

Spatial Election Explorer (SEE)

M. August Rossi, Michael Xiao, Emma Ross, Shu Luan

University of Chicago

May 20, 2026

Abstract. We introduce the Spatial Election Explorer (SEE), an interactive web application for simulating ranked-choice elections on the Euclidean plane. By encouraging users to iteratively build, experiment, and explore, SEE helps them develop visual intuition about the properties and behavior of voting rules, while grounding these concepts in real political opinion survey data.

1 Introduction

Imagine a scenario in which the members of a group have each ranked a set of alternatives from most preferred to least preferred. How should this set of rankings be consolidated to a single ranking or to a set of winning alternatives that fairly represents the entire group’s preferences? The most familiar setting for this problem is an election, where voters rank candidates to choose their representatives, but the problem setup applies much more broadly: consider a set of friends deciding what movie to watch or restaurant to visit; a committee choosing what candidates to shortlist or hire; or a panel of judges who have ranked competitors in ice skating or ballroom dance.

There are many algorithms (called *voting rules*) for aggregating these group preferences: how should the group choose which to use? Deliberation about this decision might involve invoking the details of the algorithms, or making appeals to axiomatic properties that such algorithms may or may not satisfy. Non-experts may find such arguments to be overly complex, too abstract, or simply confusing. Methods to visualize the behavior of voting rules can bridge this communication gap and help voters develop intuition about the differences between algorithms.

The core objective of our project is to employ an established election visualization technique, the two-dimensional spatial model, as the basis of an interactive election simulator. Allowing users to build, experiment, and iterate upon their own simulations builds intuition in ways that a set of static plots cannot. Combining visualization with iterative experimentation allows users to both *see* the properties of the various algorithms and to learn the conditions under which these properties emerge.

Our Methodology. To facilitate visualization, we adopt a two-dimensional spatial model of voters’ preferences. Voters and candidates are jointly embedded in the Euclidean plane, and voters rank candidates in order of increasing distance. Voters therefore prefer candidates that are closer, or “more similar”, to themselves. This model is common in both theoretical [1] and empirical [2] research in computational social science. Further, it has a natural interpretation: the position of voters and can-

didates might represent their positions on two political issues or on two ideological axes.

In the Spatial Election Explorer (SEE), voters and candidates can be placed on the plane by hand, by sampling from the available distributions, or by mapping survey data to positions on ideological axes. Their positions determine a unique collection of rankings, which can be used as input to any of various voting rules. The winners are displayed, providing the user a visual description of how the distribution of winners differs according to the choice of voting rule.

Audience. The intended audience for our application is threefold:

1. Computational social choice researchers can experiment with various voting rules, testing and refining the intuition they already have for those methods. Visualization may also suggest patterns of behavior in these voting rules that can be axiomatized and studied formally.
2. Students studying social choice theory can use the tool to develop intuition for the behavior of voting rules that supplements their learning in the classroom.
3. Policymakers and policy advocates considering ranked-choice voting can use SEE to compare voting rules and build intuition about how outcomes vary.

Related Work. Our work is primarily inspired by Elkind et al. [3], whose experiments generated plots capturing the distribution of winning candidates under various multi-winner voting rules and across varying underlying distributions of voters and candidates. SEE extends this approach by making it interactive and incorporating real-world survey data.

Another visualization approach is presented by Szufa et al. [4], who created visual “maps” of preference profiles by computing swap distances between ballots and applying the multidimensional scaling (MDS) algorithm. Extending this method to an interactive application would require some external method for generating election data; the spatial model, on the other hand, consolidates the generation and visualization steps because the position of points *determines* the voters’ rankings.

2 Preliminaries

Spatial Model of Elections. Each voter and candidate has a position in $\mathbb{R}^2$. The pairwise Euclidean distances are computed between all voters and all candidates. A voter’s ballot ranks candidates in order of increasing distance. A

collection of such rankings is called a *preference profile*.

Voting Rule. In this context, a *voting rule* is a function that takes as input a preference profile and returns a set of one or more winning alternatives.

Single-Winner Voting Rules.

Plurality. The candidate with the most first-place votes wins.

Borda. Each candidate’s score is the total count of candidates ranked below them, summed across all ballots. The candidate with the highest score wins.

Instant Runoff Voting (IRV). First-place votes are tallied. The candidate with the fewest votes is eliminated; their supporters’ votes transfer to their next favorite remaining candidate. Repeat until only one candidate remains—they are the winner.

Multi-Winner Voting Rules. Let k be the number of winners, i.e., these rules elect a committee of size k .

Single Non-Transferable Vote (SNTV). First-place votes are tallied. The k highest-scoring candidates are elected.

Bloc Plurality. Each candidate’s score is the number of ballots in which they appear in the top k positions. The k highest-scoring candidates are elected.

k -Borda. The k candidates with the highest Borda scores are elected.

Single Transferable Vote (STV). Let n be the number of voters. First-place votes are tallied to give initial candidate scores. Iterate the following process: if any candidate’s score s is greater than $q = \lfloor \frac{n}{k+1} \rfloor$, elect them, reweight each supporting voter’s ballot by a factor of $\frac{s-q}{s}$, and transfer their support to their next-favorite remaining candidate. If no candidate is above the threshold, eliminate the candidate with the lowest score and redistribute their support.

Only one candidate can be elected at a time: if multiple candidates are simultaneously above threshold, the candidate with the greater score is elected and their support is redistributed before any other candidates are elected.

3 Application Design

SEE provides an interactive plot alongside a sidebar that guides users from first contact—via a tutorial—through placing voters and candidates, running elections under various voting rules, and optionally grounding the simulation in real political survey data.

3.1 Tutorial

The tutorial serves to orient the user, introduce the voting rules, and set user expectations for the allowable interaction patterns in the application.

It begins by defining the spatial model, contextualizing the axes as representing a policy or ideology space, and emphasizing the key assumption: only the distance between voters and candidates matters in determining how voters will fill out their ballots.

Next, it explains the application and the primary ways of using it via a series of examples that introduce Plurality, Borda, IRV, and STV. At each step, the tutorial shows an example plot and asks the user to make a prediction of which candidate will win.

As users walk through the examples in the tutorial tab, they will learn about the "spoiler effect" in plurality voting systems, and how Instant-Runoff voting (IRV) avoids this effect. Next, a lesson on how Borda scores are calculated and how those scores result in electing centrist "consensus" candidates. Lastly, within the multi-winner setting, users discover that Single Transferable Vote (STV) can elect a more representative set of candidates than Bloc Plurality.

3.2 Input

On the plot, voters are blue dots, and candidates are orange stars. There are four primary methods for placing voters and candidates on the plot.

- **Manual Input.** In the upper left, users can click either the "+candidate" or "+voter" button, or engage these menus by typing 'c' or 'v' on the keyboard, respectively. Then users can click anywhere in the plot to add a voter or candidate point at that location. To exit, users right click or press 'esc' to exit. Note that within the "Survey" tab, the add voter menu is disabled, as each voter point represents a real survey respondent.
- **Synthetic Sampling.** Within the "Synthetic" tab, the user can choose one of the available distributions. The distribution center is set by dragging it on the plot, or through the input boxes. The distribution parameters adjust with slider(s) or through inputting values to the right of the slider(s). Then users then selects the type of point, voters or candidates, to be randomly sampled from the specified distribution, and inputs the desired count. Clicking "add data" adds these points to the plot. As in [3], the supported distributions are the Gaussian, the Uniform Rectangle, and the Uniform Disc.
- **Survey Data.** Upon selecting the "Survey" tab in the sidebar, the user selects two political issue topics to serve as the axes of the plot. Survey responses are plotted using a weighted average and move through the space as users re-rank the survey questions or change the axes topics. Candidates that map to the survey questions can either be added randomly (responses to all questions are randomized), or through the user manually answering each survey question themselves to create a 'Profile candidate'. Details

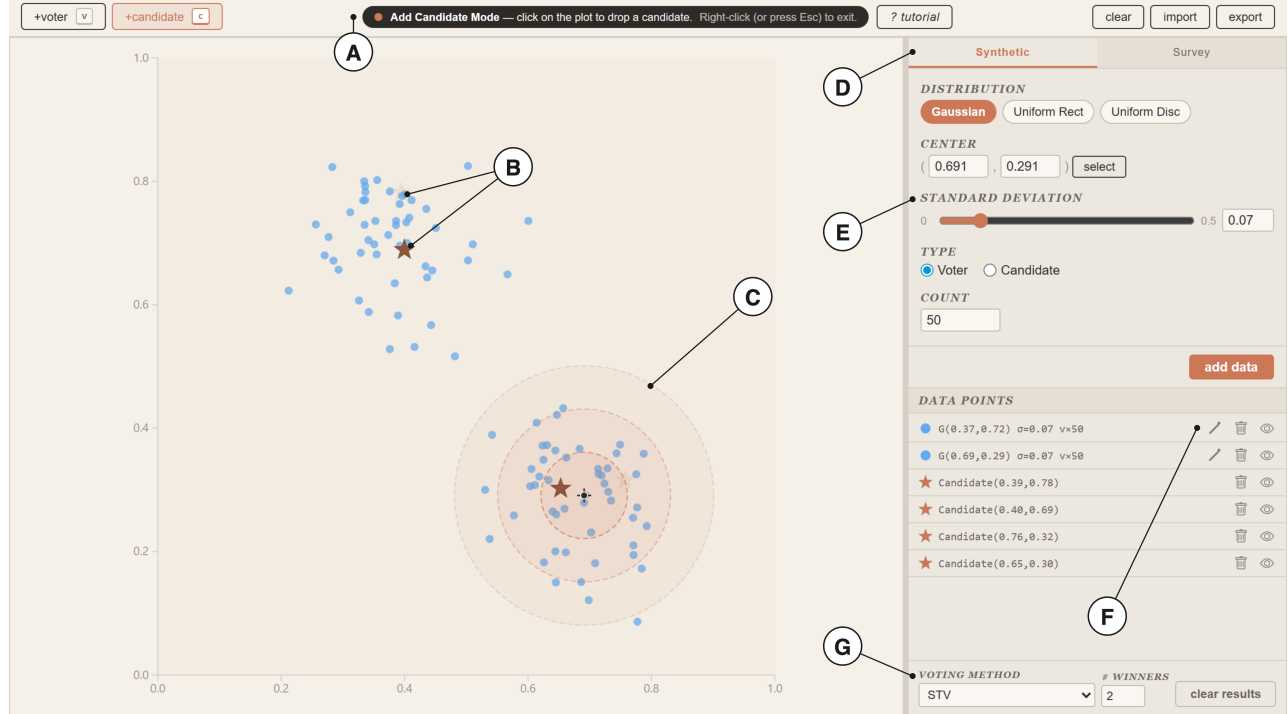


Figure 1: The Spatial Election Explorer (SEE) synthetic data interface. Voters (blue points) and candidates (orange stars) are embedded in an abstract 2D election space (unitless, $[0, 1]$); the interface lets the user place points manually, sample from synthetic distributions, load survey-grounded data, manage stacked data layers, and run an election under a chosen voting rule (§3.2)(§2). (A) Top bar: add voters or candidates by hand, launch the tutorial, import or export the current configuration. (B) Winners are rendered at full opacity; eliminated candidates are dimmed in place. (C) Gaussian sampling distribution, drawn as concentric $1\sigma/2\sigma/3\sigma$ contours. (D) Tabs switch between synthetic (§3.2) and survey data exploration (§3.3). (E) Synthetic distribution configuration. (§3.2). (F) Each manual point or sampled distribution is a stackable, editable layer. *Pencil icon*: edit distribution settings, click update to resample points. *Trash icon*: delete these point(s). *Eye icon*: hide this layer from the plot and election run. (G) Choose voting rule (§2) and number of winners, then run the election.

on the weighted average and topic selection are described in Section 3.3.

- **Data Upload.** If a user has exported a JSON file of the candidate and voter points, or survey respondent points, from a previously-generated plot, they may upload that data.

3.3 Survey Data

The Fall 2014 Simon Poll [5], conducted by the non-partisan Paul Simon Public Policy Institute, interviewed 1,006 registered voters across Illinois from September 23–October 15, 2014. The survey asks respondent for demographic details—such as sex, race, income level, education level; their plans for voting in the upcoming election; and for their opinions on political issues including government reform, LGBTQ+ rights, abortion rights, gun control laws, etc.

The survey code book lists survey questions under topics, such as 'Illinois Political Reform' and 'Guns and Society'. We use these as a starting point to develop our

axis topics. Each topic cluster given by the codebook underwent pairwise cross-tabulation analysis: for each pair of questions within a topic, we computed the conditional probability that a respondent holding a conservative (or liberal) position on one question also held a consistent position on the other. A question was included only if one political side of pairwise conditional probabilities exceeded 50%. This led us to the three political topics we use as axes: "Role of Government", "Social Issues", and "Guns".

The questions in the survey were originally on 4-point Likert scale (strongly favor to strongly oppose), 3-point scale, and binary scale. To normalize, we recoded all scales to a 0-1 range before scoring; (0, 0.33, 0.67, 1), (0, 0.5, 1), (0,1) respectively. We use rank-sum weighting based on the user's ranking of the survey questions within each topic cluster. The weighted average of a respondent's answers to the k questions in the topic cluster determines their position on the corresponding axis, with the position changing as users re-rank survey questions.

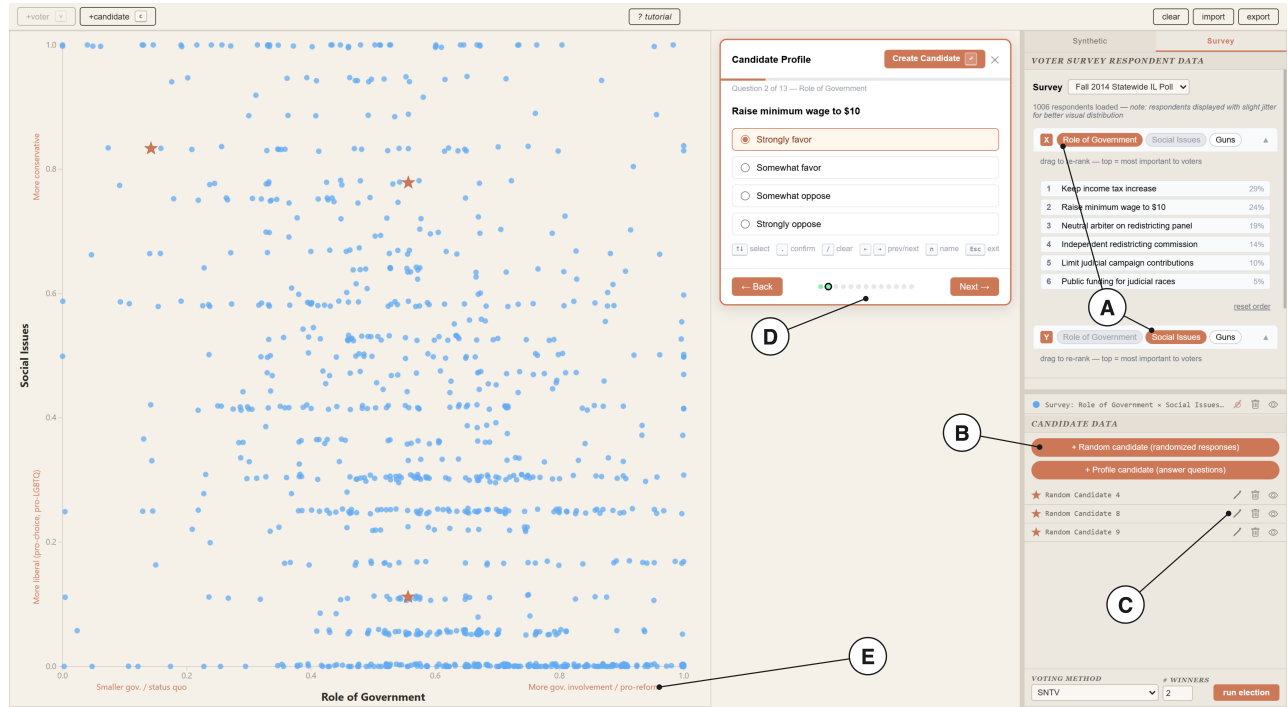


Figure 2: The survey data interface. Voters are loaded from a real political opinion poll (here, the Fall 2014 Statewide IL Poll, 1006 respondents) and projected onto two user-chosen ideology axes — in this example, Role of Government $\times$ Social Issues. Each axis is a weighted aggregate of several survey questions, re-rankable by the user. (A) Axis selection and survey question ranking panel. (B) +Random candidate creates a candidate point by randomly selecting answers to all survey questions. (C) Each candidate point is a stackable layer, editing is limited to candidates that map to the survey questions. *Pencil icon*: edit survey question responses, opens the same window as shown in (D). *Trash icon*: delete the candidate *Eye icon*: hide this candidate from the plot and election run. (D) This window opens upon clicking the +Profile candidate button, allowing users to manually specify a candidate’s responses to the survey questions to fit an ideology profile of their choice. (E) Annotations along the axis signal to users the general ideological direction of the axis.

3.4 Implementation

The application front end is built with Svelte and D3 in JS, with Vite as the server. The election algorithms are implemented in Python and compiled to Web Assembly via Pyodide so that they can be run in browser.

4 Future Directions

There are a number of natural ways to improve the application or expand its scope.

More Voting Rules. Elkind et al. [3] also examine Chamberlin-Courant (CC) and Monroe, multi-winner voting rules noted for their proportionality properties. These or any of the many methods proposed in the rich social choice literature could be worth inclusion.

Scaling to more voters and candidates. To handle more voters and candidates, two sets of changes are necessary:

1. The Python compute layer should be replaced with higher-performance code written in Rust or C++, which would significantly reduce run times for heavy

loads.

2. Rather than plotting all points, the application could, like [3], compute a fine 2D histogram, encoding frequency as color intensity. Methods for choosing representative samples from the dataset, or for estimating and visualizing density, should also be explored.

Additionally, we could replace D3 with a higher-performance visualization library that supports the WebGL API, such as Apache ECharts, deck.gl, or SciChart.js.

Improvements to the Survey Data Tab. Users would benefit from having more pre-loaded survey data to explore, and policy makers would enjoy the option to upload your own local survey data. The current implementation of the survey data plots all survey respondents, however, we could allow for sampling from the survey and visually represent uncertainty in the survey data. Additionally, providing completed survey profiles of real candidates using campaign materials would allow for analysis of real election outcomes.

References

- [1] J. M. Enelow and M. J. Hinich, *The Spatial Theory of Voting* (Cambridge Books 9780521275156). Cambridge University Press, Enero-Abr 1984.
- [2] N. Boehmer, P. Faliszewski, Ł. Janeczko, A. Kaczmarczyk, G. Lisowski, G. Pierczyński, S. Rey, D. Stolicki, S. Szufa, and T. Wąs, *Guide to Numerical Experiments on Elections in Computational Social Choice*, 2024. arXiv: 2402.11765 [cs.GT].
- [3] E. Elkind, P. Faliszewski, J.-F. Laslier, P. Skowron, A. Slinko, and N. Talmon, “What Do Multiwinner Voting Rules Do? An Experiment Over the Two-Dimensional Euclidean Domain,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, Feb. 2017.
- [4] P. Faliszewski, A. Kaczmarczyk, K. Sornat, S. Szufa, and T. Wąs, “Diversity, Agreement, and Polarization in Elections,” in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*, E. Elkind, Ed., Main Track, International Joint Conferences on Artificial Intelligence Organization, Aug. 2023, pp. 2684–2692.
- [5] Paul Simon Public Policy Institute, “Simon Poll, Fall 2014 (state wide),” Paul Simon Public Policy Institute, Southern Illinois University Carbondale, Tech. Rep. 8, 2014.